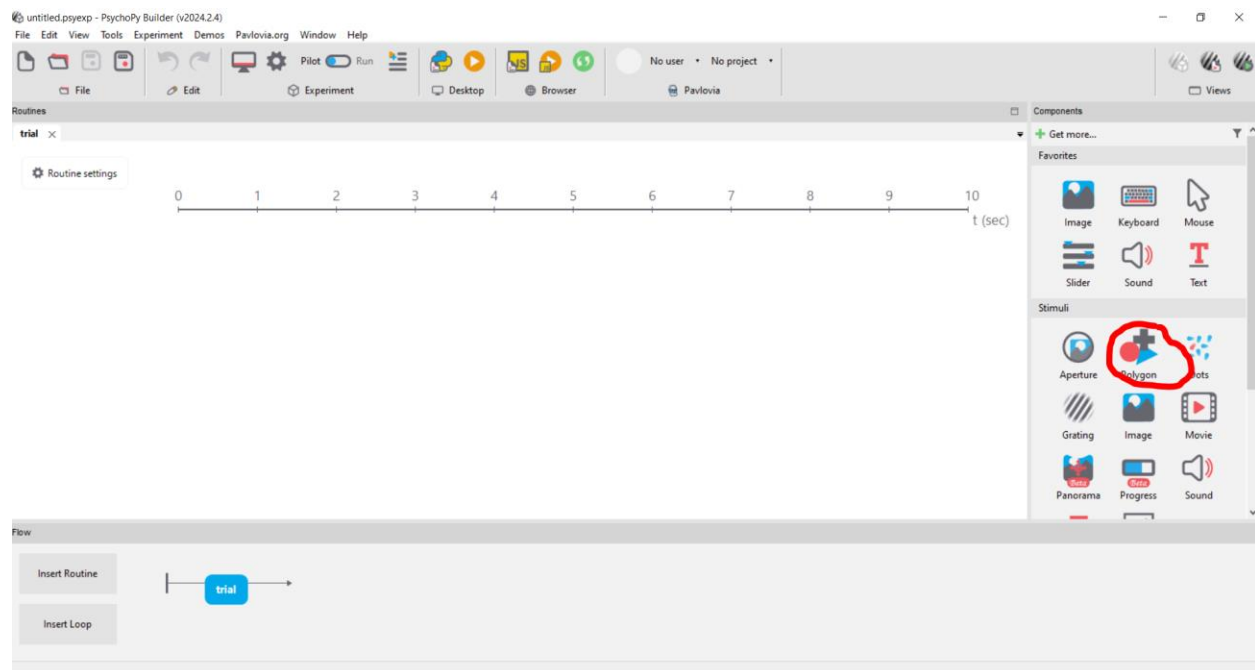


Step-by-Step: a Simple Continuous Psychophysics Experiment Designed in PsychoPy

The following is a step-by-step guide to create a approachably simplified version of the seminal tracking experiment reported by Bonnen et al. (2015) in PsychoPy.

1. Create object to be tracked

Choose “Polygon” from the right-hand column under “Stimulus”.

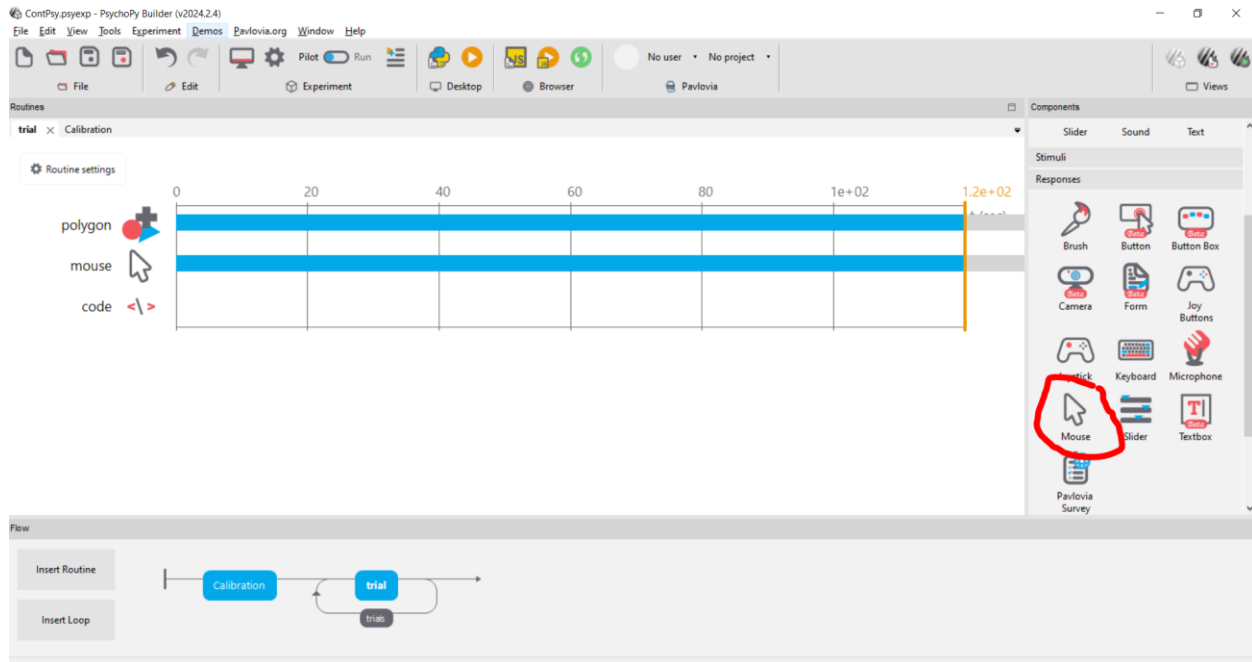


- a. Under “Basic”:
 - i. Leave “Duration” empty
 - ii. Set “Shape” to “Circle”
- b. Under “Layout”:
 - i. Set “Size” to (1,1)
 - ii. In the “Position [x,y]” field, write “pos”, and choose “set every frame” in the drop-down menu on the right.
 - iii. Set “Spatial Units” to “cm”
- c. Under “Appearance”:

- i. Write “opacity” in the Opacity field, and choose “set every repeat” in the drop-down menu on the right

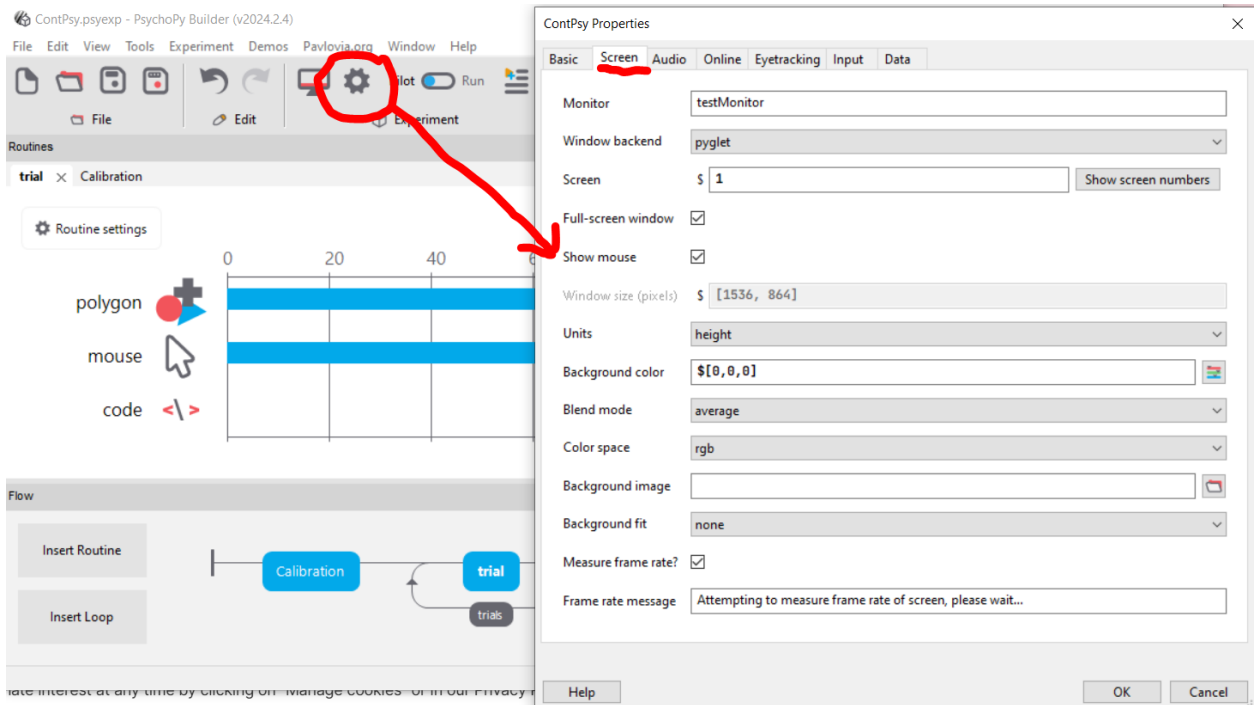
2. Create a representation of mouse cursor

Chose “Mouse” under “Responses”.



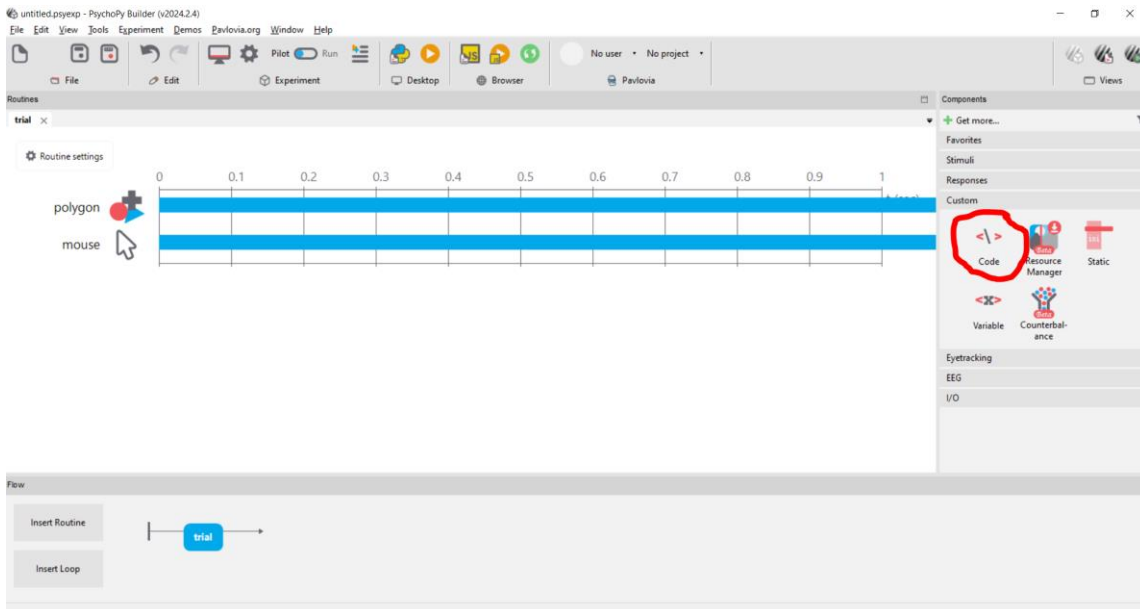
- a. Under “Basic”:
 - i. Leave “Duration” empty
 - ii. For “End Routine on Press”, choose “never”

3. Show mouse cursor during the experiment.



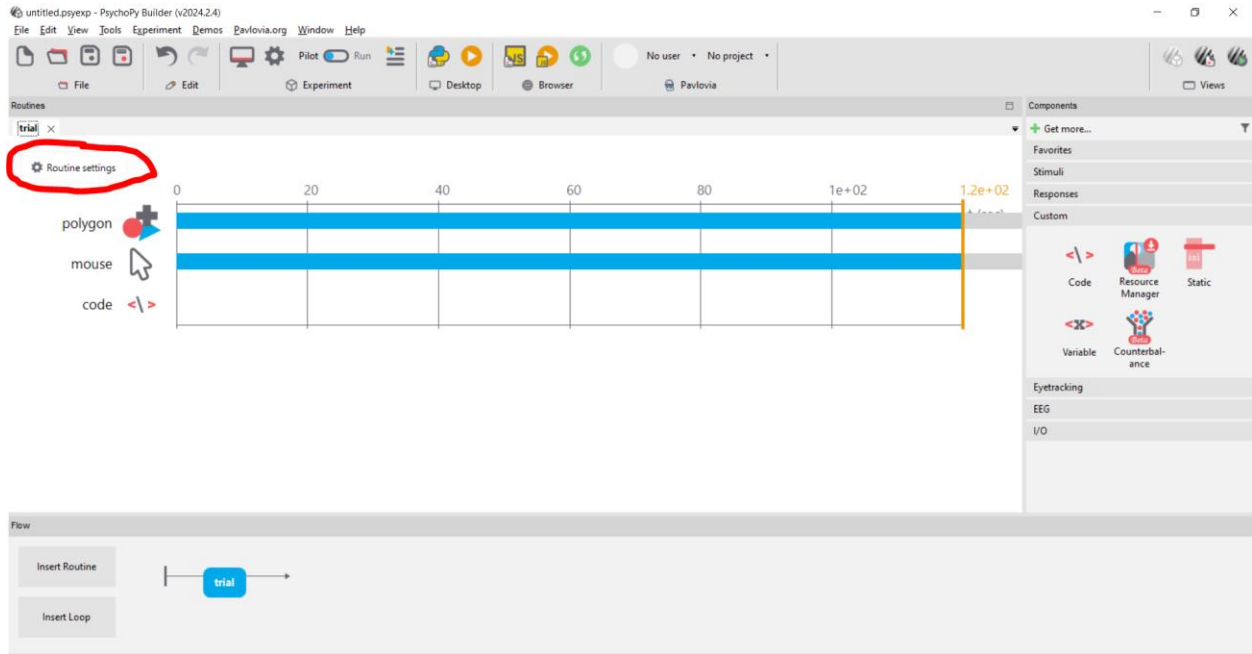
- Check “Show mouse” under “Screen” in the Experiment Settings (gear icon in the tool ribbon)

4. Create the code object



- We are returning to this later!

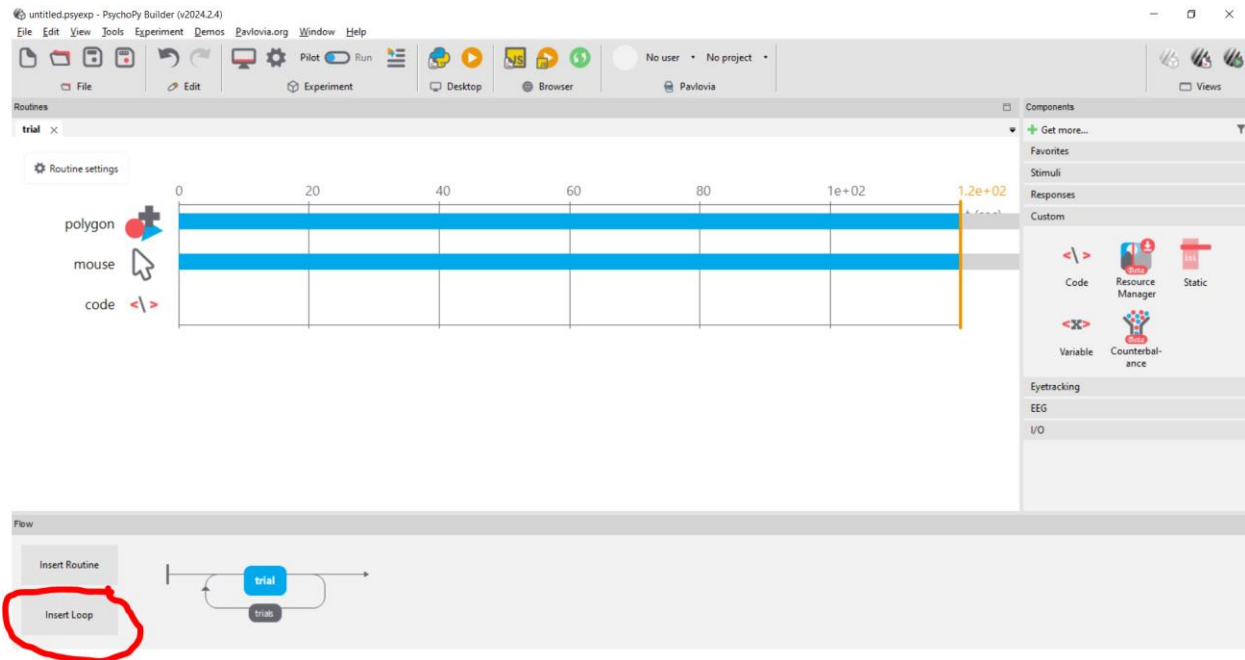
5. Configure Routine



- a. Under “Flow”:
 - i. Set Duration to 120s

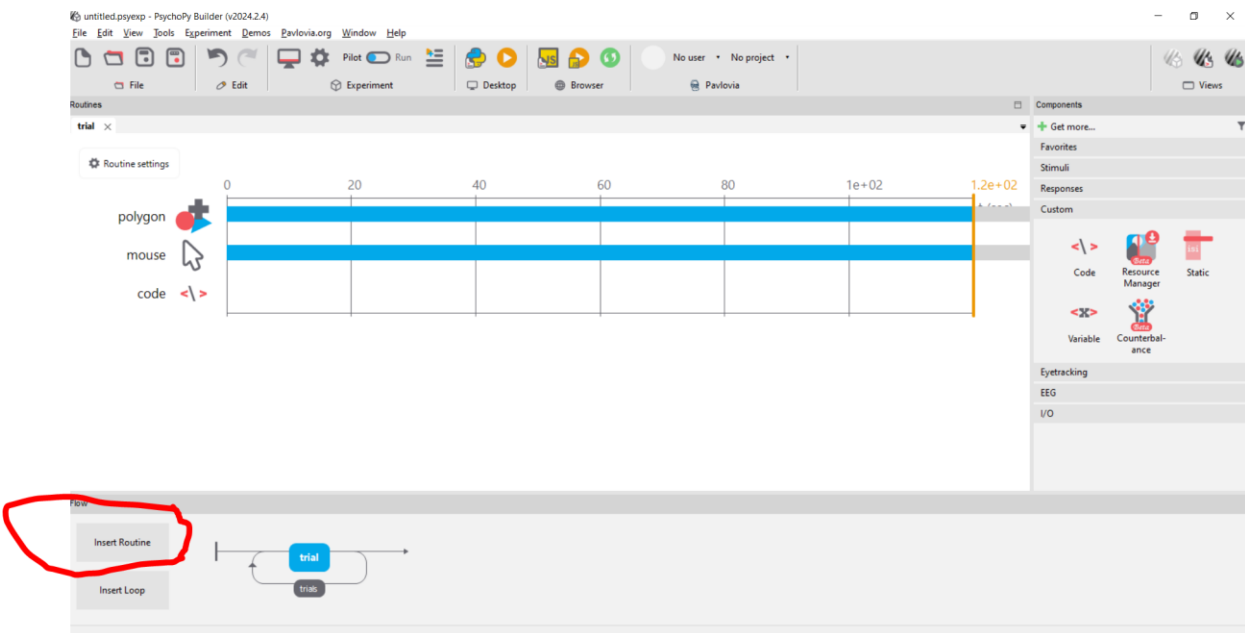
6. Create a loop

(This allows us to run the same routine twice – once with high opacity and once with low opacity. The opacity will be set using the “custom code” feature – see below.)



- a. Click on “Insert Loop” and then click on one of the two dots that appears to the left and right of the “trial” box in the bottom ribbon.
 - i. Set “Num. Repeats” to 2

7. Add calibration routine in the beginning of the experiment



- a. Insert a new Routine before “trial”. Name it “Calibration” and click on the circle to the left of “trial” to insert it.

- ## 8. Fill in code

The screenshot shows the Pavlovia.org PsychoPy Builder interface. The top menu bar includes File, Edit, View, Tools, Experiment, Demos, Pavlovia.org, Window, and Help. Below the menu is a toolbar with icons for File, Edit, Experiment, Desktop, Browser, and Pavlovia. The main window displays a routine editor for a routine named 'trial' under the 'Calibration' tab. The routine timeline shows three components: 'polygon' (0-20s), 'mouse' (20-40s), and 'code' (40-120s). The 'code' component is highlighted with a red circle. The right sidebar shows the 'Components' panel with 'Favorites' (Image, Keyboard, Mouse, Slider, Sound, Text) and 'Stimuli' (Responses, Custom, Eyetracking, EEG, I/O). The bottom panel shows the 'Flow' diagram with a 'Calibration' routine followed by a 'trial' routine loop.

- ```
#Load required packages: numpy for the random walk, time to keep track of the temporal
#aspect of the experiment
```

```
import numpy
import time
```

b. Under “Begin Experiment”, add:

```
#Here we randomize which opacity we start with.
a = [0.03,1]
opacity = numpy.random.choice(a)
```

c. Under “Begin Routine”, add:

```
#We reset the initial position of the dot at the beginning of each run/routine.
posx = 0
posy = 0
pos = (posx, posy)
```

```
#Save the time in the beginning of each run so we know how long it's been running
start = time.time()
```

d. Under “Each Frame”, add:

```
#start moving the dot only after 3 seconds
if time.time() - start > 3:
```

```
#draw the next step for the random walk for x and y directions
Addx = numpy.random.normal(loc=0,scale = 0.3)
Addy = numpy.random.normal(loc=0,scale = 0.3)
```

```
#add the next step to the current position,
#but make sure we don't go off screen (set at +- 6cm from middle of screen)
if posx + Addx < 6 and posx + Addx > -6:
 posx = posx + Addx
```

```
if posy + Addy < 6 and posy + Addy > -6:
 posy = posy + Addy
```

```
pos = (posx, posy)
```

```
#save the relevant data on each frame
thisExp.addData("time_in_run", time.time() - start)
thisExp.addData("x_coord_mouse", mouse.getPos()[0])
thisExp.addData("y_coord_mouse", mouse.getPos()[1])
thisExp.addData("x_coord_target", posx)
thisExp.addData("y_coord_target", posy)
thisExp.addData("opacity", opacity)
```

```
thisExp.nextEntry()
```

e. Under “End Routine”, add:

```
#change opacity from the one we ran first to the other one
if opacity == 0.03:
 opacity = 1
else:
 opacity = 0.03
```